

Machine Learning Assignment Week 2 by 21337470

Dataset 22-22-22

Introduction

This assignment entailed answering several questions. These questions included modelling a logistic regression and an SVM model. I have used Jupyter Notebook to do this assignment in due to its ability to run and output segments of code that I believed would be helpful when segmenting the code for each question. In this report I will explain the code that I have written and my understanding of the same.

The code index are hyperlinks that jump to the code being talked about in the code index at the end.

What is Logistic Regression?

Logistic Regression is a classification algorithm in Machine Learning. It attempts to model the relationship of the input variable with the output variable using a non-linear threshold for the output classification.

What is SVM?

SVM (Support Vector Machines) are another classification algorithm in Machine Learning. Similar to Logistic Regression it attempts to classify the data, however it uses regularization controlled by different values of C, the penalty parameter to do so. In this assignment, we have experimented with different values of C.

The Code

[Code Index : 1](#)

At the start of the code, I imported the python libraries I have used throughout the project. Numpy library was used for the function `np.column_stack` to combine the values taken from the csv file and put them in a single matrix. Pandas Library was used to read the csv file values and split the data into columns for loading in the algorithm. Matplotlib library was used for the various plots used throughout the assignment to visualize the data.

Sklearn library was used to get the various ML models used in this assignment. Logistic Regression and LinearSVC (the SVM Model) for the models, accuracy score to check the accuracy of predictions of the models and StandardScaler to scale the features before using them in the SVM.

Part A, Part (i)

[Code Index : 2](#)

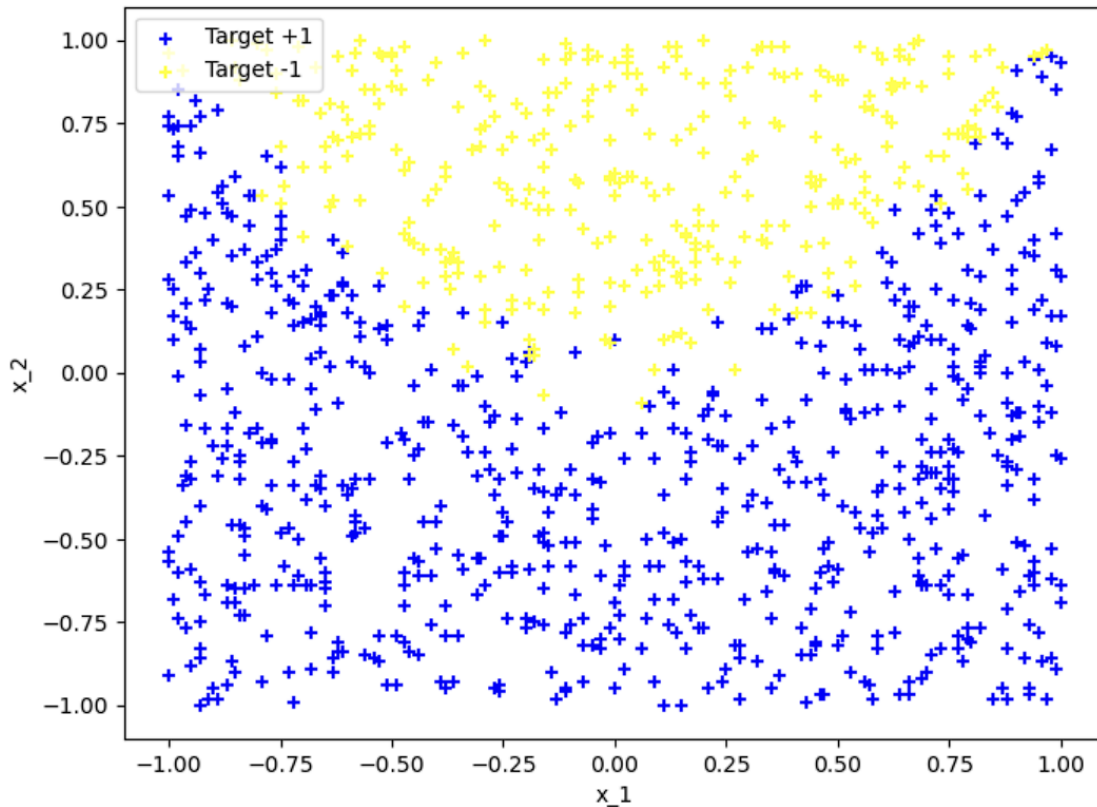
The first line `df = pd.read.csv`, we call on the pandas library and specify the CSV file that we need load the data from, the file is in the same folder as the jupyter notebook in this case so we do not need to specify the path of the file. `X1 = df.iloc` and the subsequent two lines then parse the csv file by column. 0 specifies the first column, 1 the second and so on, we load the data into the variables.

Next the `np.column_stack` is then used to combine the values from X1, X2 into a single 2D array that we can use to load the data into our ML model. After loading all the variables. I used the Matplotlib library that I imported as `plt` to draw the plot. The first line is used to specify the size of the plot using `plt.figure(figsize = (8,6))`. Next we put the data onto the plot using the `plt.scatter(X1[y == 1], X2[y == 1], marker='+', color='blue', label='Target +1')`. The `plt.scatter` specifies that we intend to create a scatter plot, the `X1[y==1]` tells to select the values of X1 where y is 1, similarly I have done this for X2. The marker specifies the type

of marker the plot will use to visualize the data, color is to specify the colour of the markers and label is for the legend of the plot. Similarly the subsequent line does this but for when the y target is -1.

plt.xlabel and plt.ylabel are then used to specify the labels of the axes of the plot. plt.legend tells it to display the legend and finally plt.show() is to display the graph.

The Graph



This graph shows the plot points, where the blue markers are for +1 values and yellow markers are for -1 values. From the plot, we can infer that both values have a quite clear distinction of boundaries as the markers do not bleed over much into one other. The blue markers are mostly concentrated below 0.25 for the y axis, from this we can reasonably infer that these values are more likely to hold negative values for x_2, similarly we can see the reverse for the yellow markers, as most of them are clustered above 0.25, we can see that these values are more likely to hold positive values for x_2.

Part A. (ii)

[Code Index : 3](#)

The first line, `log_reg = LogisticRegression()` initializes the logistic regression model into the variable. We then use the `.fit(X,y)` function on the loaded model variables to pass the feature value X, which is the 2D values of the matrix and the y target variable which indicates the type of prediction (i.e -1,+1).

Next, I used the `coeff_` and `Intercept_` function from the model variables to get the coefficient and intercept values for the model respectively and print them out. Next we load the absolute value of the coefficients to find out which one has more influence on the model. We use `coefficients[0,0]` to load the values of x1 and coefficients of x2 using `coefficients[0,1]` and we take the absolute values regardless of if the values of positive or negative.

After that we use a simple if-else statement to check which value is greater and print the output accordingly.

The Output

```
Coefficients: [[-0.11826965 -4.93868508]]  
Intercept: [1.87247119]  
X2 has more influence
```

Here we see the output of this code, we get the value of coefficients which essentially act as weights of the model. The larger the absolute value of the weight, the more influence the feature has on the model. As we can see from the values and the output, the X2 feature has more influence on the model. The intercept creates the decision boundary for the model. The intercept is affected by the feature weights as well. The decision boundary basically acts as a threshold for the model to classify the output as 0 or 1.

Part A.(iii)

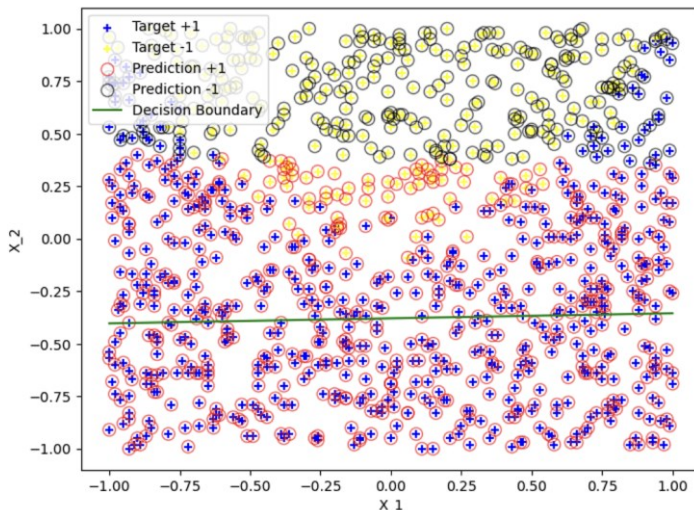
[Code Index : 4](#)

Building on the previous code, now we use `.predict` to predict the target values from the model. Next we build the original plot again using the same commands discussed earlier in the report. Using similar commands, I plotted the predictions using the 'o' marker, specifying them to be hollow using `facecolor none` command, so we can see the predictions around the training data values. We then use `x_values` and the `linspace` commands to generate 100 plot points to plot the line of the decision boundary. Then we write the equation for the decision boundary, the python code is written for the equation: $x_2 = -(w_2 + w_1 * x_1) / w_2$.

Next we use `plt.plot` again to use `x_values` and `decision_boundary` variables and specify the green color to plot the decision boundary in the plot. Similar to previous code I explained earlier in code, I used the `label`, `legend` and `show` commands from Matplotlib to label, show the legend and visualize the graph respectively. Next to display accuracy score, I called the accuracy score function I imported earlier, the `y` actual values and `y_pred`, predicted values are passed to calculate the percentage of correct predictions in their respective ratio. I multiplied the score by 100 since the accuracy score is given in decimals between 0 and 1, I multiply them by 100 in order to get a proper percentage accuracy score for better readability. And then printed the output.

The Output

Accuracy Score is 86.773547%



On the graph, we can see that the model mostly predicts the values accurately, as the red circles around the blue + markers and black circles around the yellow + markers are the accurate predictions. However, since the accuracy score is not 100% (actually 87%), we can see a few misclassifications take place, mostly near the decision boundary where there are some red circles around yellow markers and vice versa. This tells us that the model has a slight misclassification issue where the values are not as clearly separated as seen the missclassifications are mostly near the U curve mixing of the yellow and blue markers.

Part A, (iv)

From the plotting of data and its predictions, we can see the model has a good score of 87% and is able to correctly classify the data in most cases. Quoting my above inference, we can see that the model mostly predicts the values accurately, as the red circles around the blue + markers and black circles around the yellow + markers are the accurate predictions. However, since the accuracy score is not 100% (actually 87%), we can see a few misclassifications take place, mostly near the decision boundary where there are some red circles around yellow markers and vice versa. This tells us that the model has a slight misclassification issue where the values are not as clearly separated as seen the missclassifications are mostly near the U curve mixing of the yellow and blue markers.

The decision boundary is able to distinguish between both class fairly well, the boundary is linear and cannot accurately capture the data and its point as maybe a non linear boundary may, however we can see that x_2 has a larger influence on the boundary by how the boundary is almost horizontally placed. Since the boundary corresponds to an equal 50-50 split in probability, we can still see red circles above the boundary for points that still had a higher probability of being in the other class.

However, the issue is that due to the heavy influence of x_2 parameter, we can see that the decision boundary is more horizontal as well as the yellow markers being clustered near the +1 classification of the x_2 axis.

Part B, (i)

[Code Index: 5](#)

To train an SVM model, first I initialized the C_value with the values 0.001,1,100 for use with SVM and its C values.

Then I initialized a loop through the list variable (which would iterate three times for the three C values). Then we initialize the LinearSVC algorithm by calling the LinearSVC function from the imported sklearn library. We pass the parameters C from the C values to specify the C parameter for the SVM model and specifying the iterations, being 10000, I experimented with 1000 and 5000 first however I got the warning that the values were not converging. Values not converging means that our iterations are low, we need to reach a point where the loss function of our algorithm no longer significantly reduces, however if the values are not converged, that means the algorithm has not yet reached a stable solution yet.

Then I used the `svm.fit(X,y)` similar to how we used in previous model to pass through feature variable X and target variable y into the model. Then I initiaized the weights variable and got the `.coef_` value by specifying the first/0th column of the matrix that would contain it. Similarly I got the intercept of the model function by using the `.intercept_` function. Then I have used print statements to get the model parameters loaded into the variables with C hyperparameter to specify which loop they belong to.

The Output

```
Training C value: 0.001
C=0.001:
Weights: [-0.00941433 -0.46250881]
Intercept: [0.24119524]
Training C value: 1
C=1:
Weights: [-0.03934958 -1.79253197]
Intercept: [0.66970162]
Training C value: 100
C=100:
Weights: [-0.03998908 -1.81182233]
Intercept: [0.67769307]
```

Here we see that as seen before in the Regression Model, the X_2 weight is higher than X_1 . We can observe that as C values increase, the coefficient weights and intercept both increase, this is because a higher C value means less margin for error and the model trying to classify the points more tightly, leading to higher parameter values as well.

Part B. (ii)

[Code Index : 6](#)

Using the scaler variable, I added `StandardScaler()` imported from `sklearn` to scale the values so they do not disproportionally affect the model by keeping the mean 0 and standard deviation 1 and in the subsequent line, I stored these scaled values in the `X_scaled` variable.

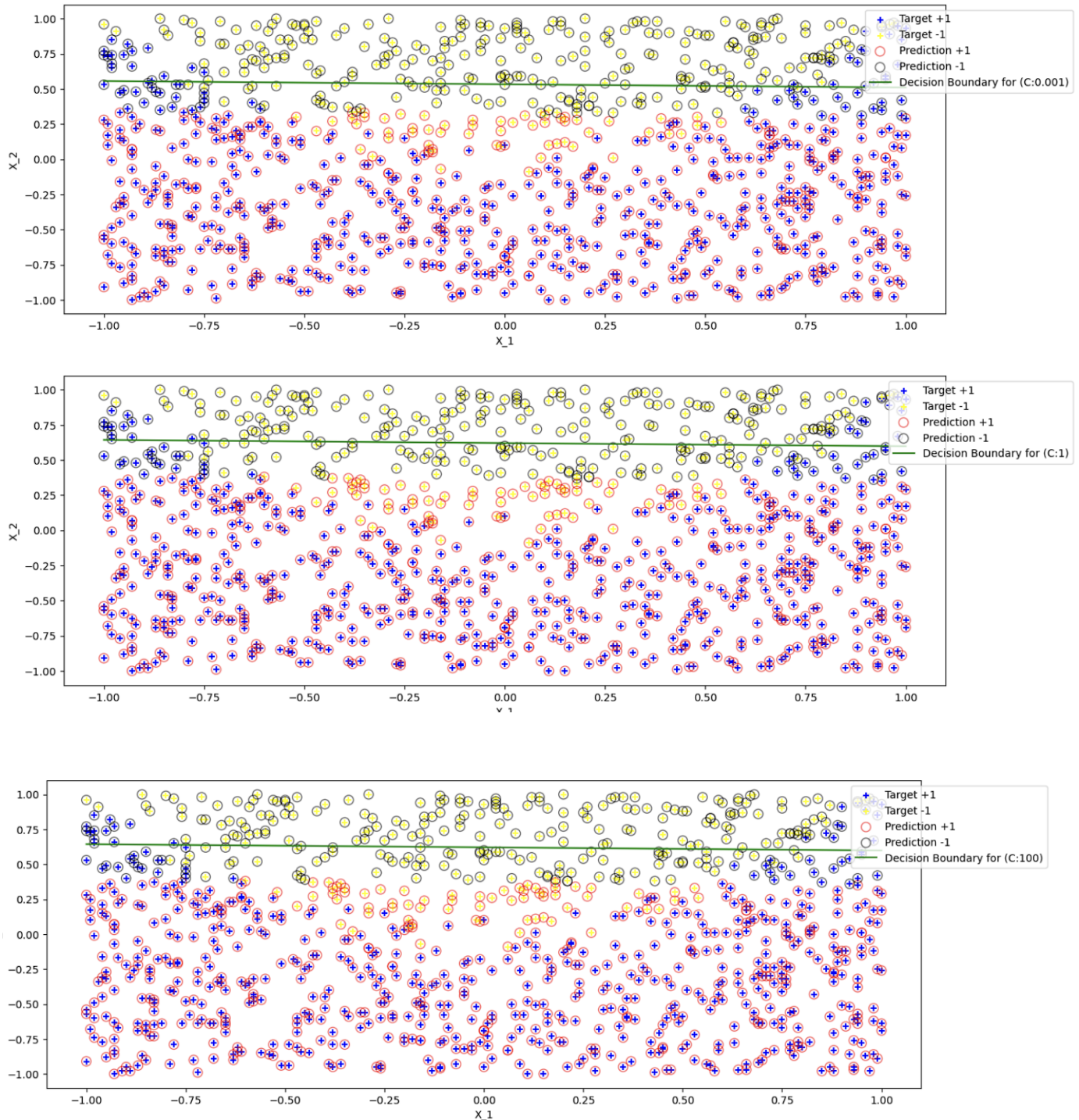
Invoking Matplotlib, Similar to before, I used `plt.figure` to specify the size of the plot. Subsequently, as before I made a loop to iterate over the `C_values`, loaded the `LinearSVC` algorithm, used the `.fit` to pass the scaled values and target parameter `y`. Then using the `predict` function to contain the predicted values.

Furthermore, then I used the formula for the decision boundary with the equation: $w_0 + w_1x_1 + w_2x_2 = 0$, where w_0 is the intercept and w_1 and w_2 are the weight coefficients.

Then using the iteration value of `i`, I created subplots using the `matplotlib` library for each value of `C` in the loop. Then, similar to before, I plotted the scatterplot with `plt.scatter` with the actual values and then plotted the predictions with the 'o' marker. The `s=80` is the size of the markers and `alpha` is the opacity which I had to use as I was facing an issue with the markers when they were plotted.

Then similarly, I plotted the decision boundary, specified the labels, legend and visualized the plot. The parameters with legend are to specify the location of the legend as the legend was being displayed over the plotted values at first.

The Output



These three plots show the output for the decision boundary with 3 different values of C , although hard to see till zoomed in, the decision boundary is getting tighter with larger values of C , confirming the earlier parameter values and showing that larger C values allow for less margin of error.

Part B. (iii)

When the C is small, the coefficient weights are smaller and the decision boundary is generally smaller. This allows for more flexibility and encompassing all data points and larger, however it can also allow for misclassification of points.

However if C is large then coefficients weights are larger as seen in the output, and it tries to correctly classify all points, however it may lead to fitting over the noise in the data as well.

Its impact on SVM predictions with a small C value is that it is better at generalizing the data however it can misclassify more points in the data. With a large value of C, the model is better at correctly classifying the data, however it may overfit and perform poorly on data it hasn't been trained on.

This happens since C is a regularization parameter, with a large value of C, the model has a very narrow margin of error and in a bid to fit every data point correctly, the model may make a very narrow margin of error and a tightly fitted decision margin, causing the model to in a sense, 'memorize' the data and perform poorly on untrained data.

Part B. (iv)

Comparing both models:

Both SVM and Logistic Regression have X_2 as the larger weight with more effect on the model.

Logistic Regression and SVM have different decision boundaries, Logistic Regression has a fixed boundary while the SVM's boundary changes accordingly to the C parameter, with the boundary getting tighter the higher the C value is. (Although in plot it might be hard to see the differences unless zoomed in, it is there and we can infer so from the intercept and coeff values as well.)

The coefficient values of the Logistic Regression model is fixed, however we can see that the coefficients change in the SVM model based on the C values, with a larger values making for higher coefficients as well and smaller C values making for smaller coefficient weights.

Part C.(i)

[Code Index = 7](#)

As specified in the question, I got the squared values of the coefficients by taking exponent of 2 and storing them in X1_squared and X_2 squared variables. To pass these values into the model, I called the np.column_stack command again and passed all X1,X2,X1_squared and X2_squared values to get the matrix containing all 4 features to use in the model.

Then, Similar to before, we get the coefficients and intercept using the .coeff_and .intercept commands, store them in their respective variables and subsequently use print functions to print the variables.

The Output

Model Parameters:

```
Weights: [[-0.01550065 -6.59730052  6.62372039 -0.99753732]]  
Intercept: [0.35845795]
```

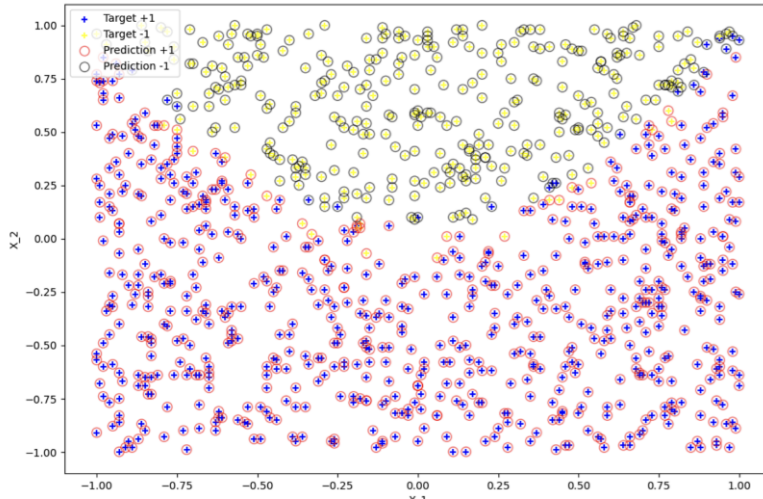
This output shows us the original coefficients and their subsequent squared values (In the order: X1,X2,X1_squared,X2_squared). As seen from previous models using this dataset, the X_2 coefficient has a much larger weight, meaning the -1 classification is more likely. Then we see the intercept value, as we can see this intercept value is smaller than the logistic regression or svm model's intercepts, which may mean a lower decision boundary as compared to the other models.

Part C.(ii)

[Code Index = 8](#)

We use the `.predict` function to get the predicted values from the earlier trained Logistic Regression Model. Then I used the Matplotlib library's `figsize`, the code I used earlier in report to plot the actual values with '+' markers and predicted values with 'o' markers, axis labels, legends and finally the `show` command to visualize the data.

The Output



From this graph, we can clearly see that the predicted values are much more accurate and although the decision boundary is not plotted yet, the predictions are non-linear and in a curve as both +1 and -1 predicted values are present on the same value in the y-axis. So instead of a straight line linear threshold boundary, there would be a curved non-linear one that is able to better encapsulate the data.

Part C.(iii)

[Code Index : 9](#)

At first, I got the accuracy of the logistic regression using the accuracy score metric by comparing the `y` and `y_pred` values against one another in a ratio so that we can see the accuracy of the predicted values against the actual values to get the accuracy score. Now to get the most frequent class for the baseline model prediction, I used `count.idxmax()` to find the highest occurrence of either +1 or -1 in the `y` value variables. Then I made a variable `y_pred_baseline` array filled with this most frequent class, meaning that the baseline model predicts the most frequent class for every data point in the dataset. Then finally, to get the accuracy score of this baseline class, I used the `accuracy_score` function again to predict the `y` value against `y_pred_baseline`.

Finally, for better readability of values, I multiplied the accuracy score values by 100 to get the answer in percentage for the accuracy rather than in decimal between 0 and 1.

The Output

```
Accuracy of Logistic Regression model: 95.9%
Accuracy of Baseline model (predicting all 1): 67.8%
```

As we can see, the Logistic Regression model with square values far outperforms the baseline model or even the previous models from part A and B. The baseline model gives the value 67.8%, though it might be low in comparison. This is a baseline model which is simply telling us the ratio of the parameters against one another, since the data is imbalanced, as we clearly know from the imbalance between the weight coefficients in the other model, we get the accuracy to be 68%.

Part C.(iv)

[Code Index = 10](#)

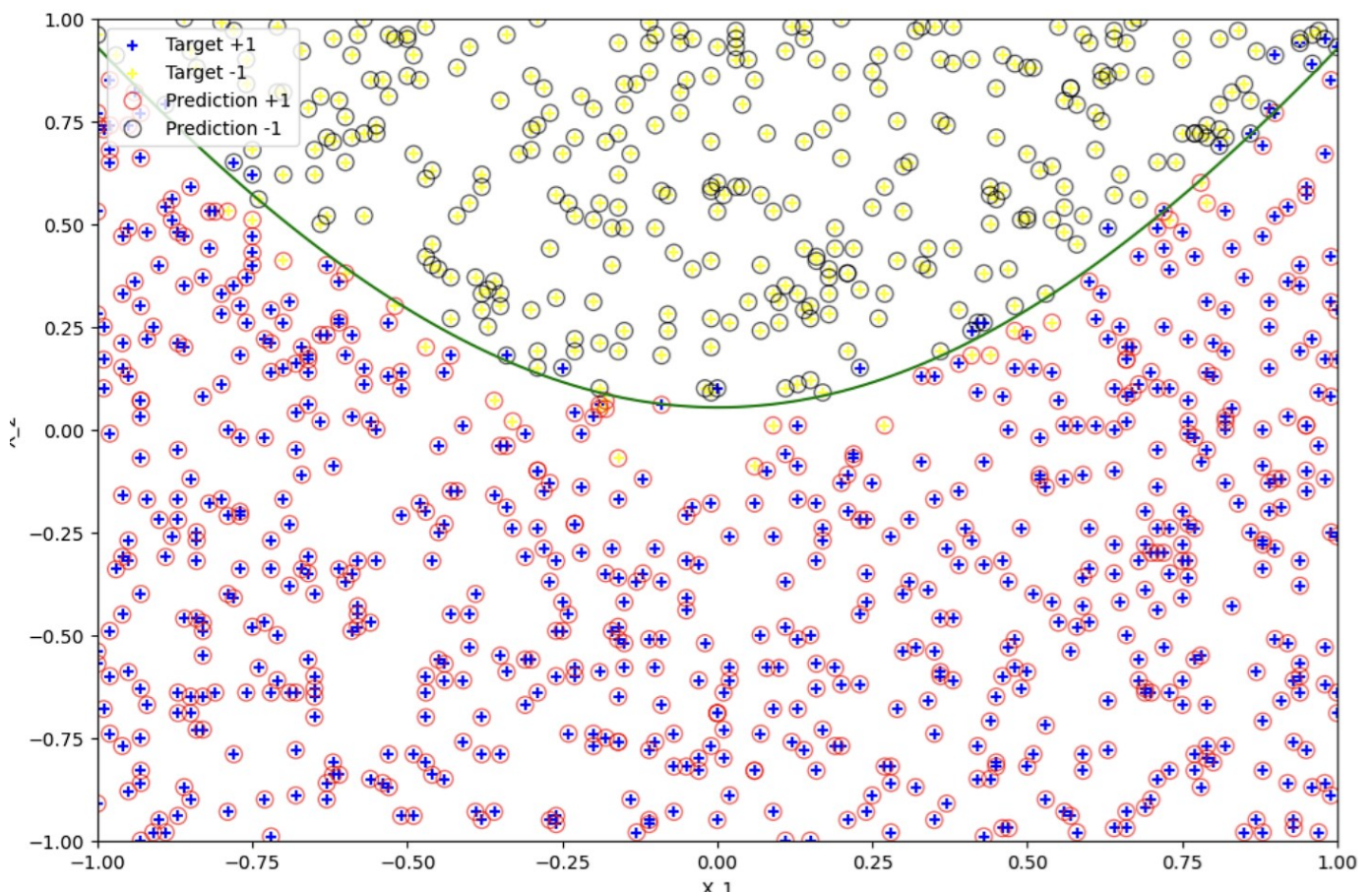
In this code, I first loaded the intercept (using the `intercept_` function) and the coefficients (using the `coef_` function) to use later on for the decision boundary equation.

Then I loaded the arrays of values into `X1` and `X2` with `linspace` to create a line of 500 points each and used the `np.meshgrid` function to create a 2D array of points covering the plot to be able to plot a non linear decision boundary on it.

After this, I used the equation : $w_0 + w_1 * X_1 + w_2 * X_2 + w_3 * X_1^2 + w_4 * X_2^2 = 0$, this builds upon the original decision boundary equation (that was linear) with a quadratic non linear terms to capture the non-linear decision boundaries. Then similar to earlier plots, I used the same commands to plot the actual and predicted plots on the graph and set the plotsize using `figsize`.

Then I plotted the decision boundary using `plt.contour`, which is able to plot contour lines, which are able to draw curves when the function has a constant value and then passed with `level=0` to specify the plot to be where the decision function is 0, since we have to make it between -1 to +1, then specified the colour of the decision boundary. Afterwards, similar to the previous plots, used `label`, `legend` and `show` functions to label the axes, legend of the plots and show to visualize the plot respectively.

The Output



As we can see from the plot, this non-linear graph almost perfectly captures the division between the points as seen by the decision boundary fitting the U-Curve of the yellow markers against the blue markers (also explains the 95% accuracy). Although there are a few misclassifications, there are far less than those of the other model and the decision boundary much perfectly fits the non linear division of markers that the linear straight line decision boundaries could not encapsulate.

CODE INDEX

1.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.svm import LinearSVC
from sklearn.preprocessing import StandardScaler
```

2.

```
df = pd.read_csv("data.csv")
X1 = df.iloc[:, 0]
X2 = df.iloc[:, 1]
y = df.iloc[:, 2]
X = np.column_stack((X1,X2))
plt.figure(figsize=(8, 6)) #Figsize determines the size of the plot
plt.scatter(X1[y == 1], X2[y == 1], marker='+', color='blue', label='Target +1')
plt.scatter(X1[y == -1], X2[y == -1], marker='+', color='yellow', label='Target -1')
# Label the axes
plt.xlabel('x_1')
plt.ylabel('x_2')
plt.legend()
plt.show()
```

3.

```
log_reg = LogisticRegression()
log_reg.fit(X, y)
coefficients = log_reg.coef_
intercept = log_reg.intercept_
print("Coefficients: ",coefficients)
print("Intercept: ", intercept)
```

```

coeff1 = abs(coefficients[0, 0])
coeff2 = abs(coefficients[0,1])
if coeff1 > coeff2:
    print("X1 has more influence")
else:
    print("X2 has more influence")

```

4.

```

y_pred = log_reg.predict(X)
#Original Plot
plt.figure(figsize=(8, 6))
plt.scatter(X1[y == 1], X2[y == 1], marker='+', color='blue', label='Target +1')
plt.scatter(X1[y == -1], X2[y == -1], marker='+', color='yellow', label='Target -1')
#Plotting Predictions
plt.scatter(X1[y_pred == 1], X2[y_pred == 1], marker='o', edgecolor='red', facecolor='none', s=80, alpha=0.6, label='Prediction +1')
plt.scatter(X1[y_pred == -1], X2[y_pred == -1], marker='o', edgecolor='black', facecolor='none', s=80, alpha=0.6, label='Prediction -1')
x_values = np.linspace(min(X1), max(X1),100)
decision_boundary = (log_reg.intercept_ +
log_reg.coef_[0][0]*x_values)/log_reg.coef_[0][1]
#plotting the boundary
plt.plot(x_values, decision_boundary,color='green',label = 'Decision Boundary')
plt.xlabel('X_1')
plt.ylabel('X_2')
plt.legend()
plt.show
accuracy = accuracy_score(y,y_pred)
accuracy *= 100
print(f"Accuracy Score is {accuracy:0f}%")

```

5.

```

C_values = [0.001,1,100]
for C in C_values:

```

```

print(f'Training C value: {C}')
svm = LinearSVC(C=C, max_iter=10000)
svm.fit(X,y)
weights = svm.coef_[0]
intercept = svm.intercept_
print(f"C={C}:")
print(f"Weights: {weights}")
print(f"Intercept: {intercept}")

```

6.

```

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
plt.figure(figsize=(15,18))
for i,C in enumerate(C_values):
    svm = LinearSVC(C=C, max_iter =50000)
    svm.fit(X_scaled, y)
    y_pred = svm.predict(X_scaled)
    decision_boundary = -(svm.intercept_ + svm.coef_[0][0]*x_values) /
svm.coef_[0][1]
    #Subplots for every C value
    plt.subplot(3,1,i+1)
    #Plotting actual values
    plt.scatter(X1[y == 1],X2[y == 1], marker='+', color='blue', label='Target
+1')
    plt.scatter(X1[y == -1],X2[y == -1], marker='+',color='yellow', label=
'Target -1')
    #Plotting Predictions
    plt.scatter(X1[y_pred == 1],X2[y_pred == 1], marker='o',
edgecolor='red',facecolor='none',s=80, alpha =0.6,label='Prediction +1')
    plt.scatter(X1[y_pred == -1],X2[y_pred == -1], marker='o',
edgecolor='black',facecolor='none',s=80, alpha=0.6, label='Prediction -1')
    plt.plot(x_values, decision_boundary, color='green', label=f'Decision
Boundary for (C:{C})')
    plt.xlabel('X_1')
    plt.ylabel('X_2')
    plt.legend(loc='upper right', bbox_to_anchor=(1.15, 1)
plt.show()

```

7.

```
X1_squared = X1 ** 2
X2_squared = X2 ** 2

X_extended= np.column_stack((X1,X2,X1_squared,X2_squared))

log_reg_extended= LogisticRegression()
log_reg_extended.fit(X_extended, y)

coefficients = log_reg_extended.coef_
intercept = log_reg_extended.intercept_

print("Model Parameters:")
print(f"Weights: {coefficients}")
print(f"Intercept: {intercept}")
```

8.

```
y_pred_extended = log_reg_extended.predict(X_extended)

plt.figure(figsize=(12,8))

#Plot actual values
plt.scatter(X1[y == 1],X2[y == 1], marker='+', color='blue', label='Target +1')
plt.scatter(X1[y == -1],X2[y == -1], marker='+',color='yellow', label= 'Target -1')

#Plot Predictions
plt.scatter(X1[y_pred_extended == 1],X2[y_pred_extended == 1], marker='o',
edgecolor='red',facecolor='none',s=80, alpha =0.6,label='Prediction +1')
plt.scatter(X1[y_pred_extended == -1],X2[y_pred_extended == -1], marker='o',
edgecolor='black',facecolor='none',s=80, alpha=0.6, label='Prediction -1')

plt.xlabel('X_1')
plt.ylabel('X_2')
```



```
plt.legend()
```

```
plt.show()
```

9.

```
accuracy_log_reg = accuracy_score(y,y_pred_extended)
```

```
most_frequent_class = y.value_counts().idxmax()
```

```
y_pred_baseline = np.full_like(y,fill_value = most_frequent_class)
```

```
accuracy_baseline = accuracy_score(y, y_pred_baseline)
```

```
accuracy_log_reg *= 100
```

```
accuracy_baseline *= 100
```

```
print(f"Accuracy of Logistic Regression model: {accuracy_log_reg:.1f}%")
```

```
print(f"Accuracy of Baseline model (predicting all {most_frequent_class}):  
{accuracy_baseline:.1f}%")
```

10.

```
w0 = log_reg_extended.intercept_[0]
```

```
w1,w2,w3,w4 = log_reg_extended.coef_[0]
```

```
x1_values = np.linspace(X1.min(),X1.max(),500)
```

```
x2_values = np.linspace(X2.min(),X2.max(),500)
```

```
x1_grid, x2_grid = np.meshgrid(x1_values, x2_values)
```

```
decision_boundary= w0 + w1 * x1_grid + w2 * x2_grid + w3 * (x1_grid ** 2) + w4 *  
(x2_grid ** 2)
```

```
plt.figure(figsize=(16,12))
```

```
#Plot actual values
```

```
plt.scatter(X1[y == 1],X2[y == 1], marker='+', color='blue', label='Target +1')
```

```
plt.scatter(X1[y == -1],X2[y == -1], marker='+',color='yellow', label= 'Target -  
1')
```

```
plt.scatter(X1[y_pred_extended == 1],X2[y_pred_extended == 1], marker='o',  
edgecolor='red',facecolor='none',s=80, alpha =0.6,label='Prediction +1')  
  
plt.scatter(X1[y_pred_extended == -1],X2[y_pred_extended == -1], marker='o',  
edgecolor='black',facecolor='none',s=80, alpha=0.6, label='Prediction -1')  
  
plt.contour(x1_grid, x2_grid, decision_boundary, levels=[0], colors='green')  
  
plt.xlabel('X_1')  
plt.ylabel('X_2')  
  
plt.legend()  
  
plt.show()
```